

AI POWERED INTELLIGENT HOME PLANNING & OPTIMIZATION SYSTEM

Project ID: 25-26J-201

Individual Report

Rathnasinghe S.J.R

BSc(Hons) Degree in Information Technology
Specialization in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

Individual Report

Project ID: 25-26J-201

Rathnasinghe S.J.R – IT

Supervisor: Prof. Samantha Rajapaksha

Co-Supervisor: Ms. Adya Dissanayaka

BSc(Hons) Degree in Information Technology
Specialization in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature:  _____

Date: 23/04/2026

ABSTRACT

Arranging furniture inside a bedroom sounds simple, but doing it well — respecting every wall, door, window, and clearance requirement at once — is genuinely hard, especially for someone without a design background. This report describes a system I built to solve exactly that problem. It is the Interior Space Optimization component of a larger web-based platform called the Integrated Home Design and Planning Platform, developed at the Sri Lanka Institute of Information Technology.

The core of the solution is a Genetic Algorithm (GA) that I designed specifically for bedroom layouts. Rather than placing furniture randomly, the algorithm starts with smart placements based on how bedrooms are actually arranged — bed against the main wall first, then bedside items, then storage — and improves from there over many generations. A custom fitness function scores each candidate layout across geometric checks (is everything inside the room? do pieces overlap? are doors clear?) and design quality checks (is the bed centred on the wall? is there a clear path to the bed? is the centre of the room open?). The Shapely geometry library handles all the polygon-level maths that make these checks exact rather than approximate.

I also added an optional hybrid mode where a small PyTorch neural network — trained on locally generated bedroom scenarios — adjusts the fitness score based on learned aesthetic patterns. A diversity filter then picks the three most different high-scoring layouts to show the user, rather than three nearly identical ones.

Testing across thirty bedroom configurations showed a 96.2% hard constraint satisfaction rate and a 75.3% similarity to layouts produced by a professional interior designer. The hybrid mode improved the aesthetic score by 11.9%. Most layouts were generated in under five seconds, making the tool practical for real-time use.

Keywords: *Interior Space Optimization, Genetic Algorithm, Furniture Layout, Constraint Satisfaction, Machine Learning, Computational Geometry*

ACKNOWLEDGEMENTS

Completing this research was not something I could have done alone, and there are several people I genuinely want to thank.

First and foremost, I extend my deepest appreciation to my research project supervisor, **Prof. Samantha Rajapaksha**, whose guidance throughout this project was invaluable. His feedback always pushed me to think more carefully about my design choices, and conversations with him often helped me work through technical problems I had been stuck on for days. I am particularly grateful for his patience during the more difficult stages of the implementation.

I am also sincerely thankful to my co-supervisor, **Ms. Adya Dissanayaka**, for her continued encouragement and practical suggestions. Whenever I was unsure about the direction of the work, her input helped me refocus. She made herself approachable throughout the project, and that made a real difference.

The Department of Information Technology at SLIIT provided a good environment to carry out this research — access to resources, lab facilities, and a community of people working on related problems all helped.

I also want to acknowledge my group members who worked on the other components of the platform — the land analysis, exterior space optimization, and Vastu compliance modules. Working alongside them gave me a clearer picture of how the whole system fits together, and their feedback on the integration side of things was useful.

A lot of what made this technically possible was built on open-source tools — Shapely, PyTorch, FastAPI, React, Konva.js, and Three.js among others. Credit goes to the developers and communities behind those libraries.

Finally, to my family and close friends — thank you for putting up with me during the long stretches where I was absorbed in this. Your support meant more than I can properly express here.

Rathnasinghe S. J. R
IT22908124
April 2026

TABLE OF CONTENTS

DECLARATION	1
ABSTRACT	2
ACKNOWLEDGEMENTS	3
TABLE OF CONTENTS	5
LIST OF FIGURES	8
LIST OF TABLES	9
LIST OF ABBREVIATIONS	10
CHAPTER 1: INTRODUCTION	11
1.1 Background and Motivation.....	11
1.2 Literature Survey.....	12
1.2.1 Computational Approaches to Space Planning	12
1.2.2 Genetic Algorithms in Layout Optimization	13
1.2.3 Machine Learning for Interior Design	15
1.2.4 Constraint Satisfaction in Spatial Problems.....	15
1.3 Research Gap	16
1.4 Research Problem.....	17
1.5 Research Objectives	18
1.6 Scope	19
1.7 Report Structure	19
CHAPTER 2: METHODOLOGY	20
2.1 System Architecture Overview	20
2.2 Geometric Modelling and Room Representation.....	20
2.3 Furniture Representation and Catalogues	22
2.4 Constraint Formulation	23

2.4.1 Hard Constraints	23
2.4.2 Soft Constraints and Heuristic Rules	24
2.5 Genetic Algorithm Design	26
2.5.1 Chromosome Encoding	26
2.5.2 Population Initialization	26
2.5.3 Fitness Evaluation	27
2.5.4 Selection, Crossover, and Mutation	28
2.5.5 Diversity Filtering and Elite Preservation	29
2.6 Machine Learning Surrogate Model	30
2.6.1 Model Architecture	30
2.6.2 Feature Engineering	31
2.6.3 Training Procedure	32
2.7 Hybrid Optimization Mode	33
2.8 Commercialization Aspects of the Product	33
2.9 Testing and Implementation	34
2.9.1 Unit Testing	34
2.9.2 Integration Testing	35
2.9.3 Frontend Implementation	35
CHAPTER 3: RESULTS AND DISCUSSION	37
3.1 Experimental Setup	37
3.2 Optimization Performance Results	37
3.3 Research Findings	39
3.4 Discussion	40
3.5 Summary of Each Student's Contribution	41
CHAPTER 4: CONCLUSION	43
4.1 Summary of Contributions	43

4.2 Limitations	44
4.3 Future Work	44
REFERENCES.....	46
APPENDICES	48
Appendix A: Sample Dataset Schema	48
Appendix B: API Request/Response Specification	48
Appendix C: Fitness Function Pseudocode	49

LIST OF FIGURES

<u>Figure 1.1</u>	<u>System context diagram of the Integrated Home Design and Planning Platform</u>
<u>Figure 1.2</u>	<u>Taxonomy of computational layout optimization approaches</u>
<u>Figure 1.3</u>	<u>Proposed research framework and component overview</u>
<u>Figure 2.1</u>	<u>High-level system architecture of the Interior Space Optimization service</u>
<u>Figure 2.2</u>	<u>Room polygon representation with door and window vectors</u>
<u>Figure 2.3</u>	<u>Hard constraint zones: boundary, clearance, and walkway buffers</u>
<u>Figure 2.4</u>	<u>Chromosome encoding scheme for GA furniture placement</u>
<u>Figure 2.5</u>	<u>Population initialization flow with priority-based placement</u>
<u>Figure 2.6</u>	<u>Fitness score decomposition diagram</u>
<u>Figure 2.7</u>	<u>Genetic algorithm workflow — selection, crossover, mutation, repair</u>
<u>Figure 2.8</u>	<u>Diversity filtering process for top-3 layout selection</u>
<u>Figure 2.9</u>	<u>InteriorFitnessModel neural network architecture</u>
<u>Figure 2.10</u>	<u>Hybrid optimization mode workflow</u>
<u>Figure 2.11</u>	<u>Frontend module workflow — room drawing to layout output</u>
<u>Figure 2.12</u>	<u>2D Konva canvas layout visualization — sample optimized bedroom</u>
<u>Figure 2.13</u>	<u>3D Three.js visualization of optimized bedroom layout</u>
<u>Figure 3.1</u>	<u>Fitness score distribution across 30 test rooms</u>
<u>Figure 3.2</u>	<u>Execution time vs. furniture count</u>
<u>Figure 3.3</u>	<u>GA vs. hybrid mode fitness score comparison</u>
<u>Figure 3.4</u>	<u>Layout diversity of top-3 outputs — sample room</u>

LIST OF TABLES

<u>Table 1.1</u>	<u>Comparison of related interior layout optimization systems</u>
<u>Table 2.1</u>	<u>Bedroom room dimension constraints and clearance parameters</u>
<u>Table 2.2</u>	<u>Standard furniture categories and dimension ranges</u>
<u>Table 2.3</u>	<u>Hard constraint parameters and thresholds</u>
<u>Table 2.4</u>	<u>Soft constraint scoring weights and conditions</u>
<u>Table 2.5</u>	<u>Genetic algorithm configuration parameters</u>
<u>Table 2.6</u>	<u>Fitness function component breakdown</u>
<u>Table 2.7</u>	<u>InteriorFitnessModel layer configuration</u>
<u>Table 2.8</u>	<u>Feature vector description (9 features)</u>
<u>Table 2.9</u>	<u>InteriorFitnessModel training hyperparameters</u>
<u>Table 2.10</u>	<u>Execution time benchmarks by furniture count</u>
<u>Table 3.1</u>	<u>Optimization results for standard bedroom sizes (n=30 rooms)</u>
<u>Table 3.2</u>	<u>GA vs. hybrid mode performance metrics</u>
<u>Table 3.3</u>	<u>Layout similarity scores against professional reference designs</u>

LIST OF ABBREVIATIONS

Abbreviation	Full Form
2D	Two-Dimensional
3D	Three-Dimensional
API	Application Programming Interface
CSP	Constraint Satisfaction Problem
GA	Genetic Algorithm
GAN	Generative Adversarial Network
GNN	Graph Neural Network
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation
ML	Machine Learning
MOCO	Multi-Objective Combinatorial Optimization
MSE	Mean Squared Error
REST	Representational State Transfer
RL	Reinforcement Learning
SA	Simulated Annealing
SLIIT	Sri Lanka Institute of Information Technology
UI	User Interface
UX	User Experience

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Anyone who has moved into a new home knows the problem. You have a room, a bed, a wardrobe, a couple of bedside tables, and maybe a dressing table — and you need to fit them together in a way that actually works. The door needs to open freely. You need to be able to walk to the bed in the dark without hitting anything. The wardrobe needs space in front of it so you can actually open it. And ideally, the whole thing should not look as though the furniture was just pushed to wherever it fit first.

For most people, this process involves moving things around by hand, realising something does not work, and moving them again. It takes time, it is physically demanding, and there is no guarantee you end up with the best arrangement — just one that is good enough. Professional interior designers solve this problem well, but hiring one is expensive and not realistic for most first-time homeowners.

That gap is what motivated this research. The Interior Space Optimization component, which I developed as part of the Integrated Home Design and Planning Platform at SLIIT, tries to close it. Given a room shape, door and window positions, and a chosen set of furniture items, the system automatically generates three different layout options — each geometrically valid, ergonomically sound, and reasonably well-arranged — within a few seconds, through a web browser.

The timing for this kind of tool is good. Sri Lanka's residential construction sector has grown steadily as urban populations rise and more young families buy their first homes. The cost of professional design services puts them out of reach for many of these buyers. A free, browser-accessible tool that can suggest sensible furniture arrangements has real practical value in this context.

From a technical standpoint, the problem is harder than it looks. Furniture positions are continuous variables — a piece can sit anywhere within the room, not just on a grid. Each item adds three degrees of freedom: x position, y position, and rotation. Constraints interact with each other in complex ways. And there is no single correct answer; many valid arrangements exist. These properties make the problem well-suited to evolutionary search methods, and a genetic algorithm in particular.

This dissertation describes the three main technical things I built: a domain-adapted genetic algorithm for bedroom layout generation, a geometric constraint checker built on the Shapely library, and an optional neural network surrogate model (implemented in PyTorch) that adds a learned aesthetic score on top of the rule-based fitness function.

[Figure 1.1: System Context Diagram of Integrated Home Design and Planning Platform — Insert Screenshot Here]

Figure 1.1: System context diagram showing the Interior Space Optimization component within the broader platform

1.2 Literature Survey

Research on computational furniture layout and space planning goes back several decades. Understanding where earlier work succeeded and where it fell short helped shape the design decisions in this project.

1.2.1 Computational Approaches to Space Planning

The earliest attempts at automating interior layout used rule-based expert systems — essentially large collections of if-then design rules encoded in software. Flemming and Bhavnani (1994) built one of the earlier such systems for architectural layout. These approaches worked reasonably well when the room geometry was simple and the rules covered the situation, but they were brittle. Any room shape or furniture combination the rules did not anticipate caused them to fail or produce poor results.

Constraint satisfaction methods came next and offered a more principled way to frame the problem: define the constraints formally, then find an assignment of positions and orientations that satisfies them all. Borning et al. (1997) applied incremental constraint propagation to geometric layout problems with some success. The limitation is that these methods focus on finding any feasible solution rather than a good one — they do not naturally optimise across multiple competing quality criteria at the same time.

Simulated annealing (SA) was another direction explored in this era. Yu and Kalay (2003) used SA for furniture placement and showed it could satisfy both geometric and functional constraints within reasonable computation time. The downside is that SA follows a single solution trajectory — it finds one good layout, not several diverse ones.

More recent approaches include the probabilistic generative model by Merrell et al. (2011), which learns spatial relationships from a corpus of existing room layouts and applies them to generate new ones. This produces naturalistic-looking arrangements, but requires large training datasets and provides no formal guarantee that the output will satisfy all hard constraints.

Deep learning approaches like LayoutGAN (Li et al., 2019) go further, using generative adversarial networks to synthesise plausible layouts from learned data distributions. Visually these can look impressive, but they share the same weakness: no constraint guarantees, and heavy dependence on large labelled datasets that may not reflect local design norms.

1.2.2 Genetic Algorithms in Layout Optimization

Genetic algorithms were first described by Holland (1975) and brought to a wider audience through Goldberg's (1989) textbook. The core idea — maintaining a population of candidate solutions and improving them through selection,

recombination, and mutation — turns out to be a natural fit for layout problems, where the solution space is large, irregular, and not amenable to gradient-based search.

Michalek et al. (2002) applied a GA to architectural floor plan layout and found that domain-specific chromosome representations outperformed generic ones significantly. This insight directly influenced how I encoded the chromosome in this system — rather than using a generic binary string, each gene encodes the (x, y, rotation) tuple for one specific furniture item.

Barbosa and Feijó (1995) extended earlier work on spatial layout optimisation to include constraint repair operators in the GA — procedures that fix infeasible offspring after crossover or mutation rather than simply discarding them. This is important in practice because purely random genetic operators frequently produce placements that violate basic geometric constraints. I adopted a similar repair step in this system.

Panahi and Kamari (2020) specifically tackled residential floor plan optimisation using a GA, incorporating both functional and proportion rules. While their work operated at the room-level rather than the furniture-level, it confirmed that domain-specific initialization heuristics and multi-objective fitness formulations are the right approach for residential design contexts.

Sareni and Krahenbuhl (1998) raised a problem that became directly relevant to this project: evolutionary optimisers tend to converge on a cluster of similar solutions, so if you just take the top N results, you end up showing the user N nearly-identical layouts. Their fitness sharing and crowding mechanisms address this. I approached the same problem differently, through a post-evolution diversity filtering step described in Section 2.5.5.

[Figure 1.2: Taxonomy of Computational Layout Optimization Approaches — Insert Screenshot Here]

Figure 1.2: Classification of computational approaches to interior layout optimization

1.2.3 Machine Learning for Interior Design

Over the past decade, machine learning has entered the interior design space in several ways. Style classification networks (Lim et al., 2018) showed that visual aesthetic properties — things that are genuinely difficult to write down as rules — can be learnt from labelled image datasets. Layout generation networks like Ritchie et al. (2019) and graph neural network approaches like Wang et al. (2021) pushed further, trying to generate entire room arrangements from learned distributions.

The approach I took is somewhat different. Rather than using ML to generate layouts from scratch, I used it as a surrogate fitness function — a learned scorer that complements the rule-based geometric fitness. Jin (2011) surveys this idea in depth in the surrogate-assisted evolutionary algorithm literature, showing that a learned approximation of the fitness function can substantially improve optimization quality when the true function is expensive or incomplete. In this case, the 'true' function is a professional designer's aesthetic judgement, which is hard to fully encode as rules.

State-of-the-art systems like HouseGAN++ (Nauata et al., 2021) produce impressive room-level layouts using graph-conditioned GANs, but extending them to furniture-level placement within specific room geometries, with hard constraint guarantees, remains an open problem. This is the gap the current system directly targets.

1.2.4 Constraint Satisfaction in Spatial Problems

The Shapely library (Gillies et al., 2007) is the geometric backbone of the constraint checking in this system. It implements the OpenGIS specification for 2D geometric operations — containment tests, intersection area computations, buffer generation, nearest-point calculations — and its Python API maps cleanly onto the constraint definitions I needed to implement.

The specific clearance values used in the system — 0.9m for door buffers, 0.3m for window clearance, 0.7m for walkway width — are grounded in accessibility design research. Panero and Zelnik (1979) documented anthropometric furniture clearance requirements that have become the standard reference in interior design practice globally. The walkway check I implemented, using Shapely LineString intersection operations, is a simplified version of the path-existence problems studied in robot motion planning (LaValle, 2006), adapted to the lower-precision requirements of bedroom layout.

The multi-objective nature of the problem — simultaneously optimising for space utilisation, accessibility, and aesthetics — places it in the domain of multi-objective combinatorial optimisation (MOCO), covered comprehensively by Ehrgott (2005). The weighted aggregation approach I use in the fitness function is one of the standard approaches in MOCO for converting a multi-objective problem into a single-objective one amenable to standard evolutionary search.

1.3 Research Gap

Looking at the existing literature, five gaps stand out clearly:

Most computational layout systems treat rooms generically. They do not encode the particular semantic hierarchy of bedroom furniture — the fact that the bed is the primary item that everything else positions itself around, or that bedside tables should be symmetrically accessible on each side of the bed. A system that understands these bedroom-specific relationships will naturally produce better bedroom layouts.

Virtually no published system returns multiple diverse alternatives to the user. They optimise and return a single best layout. But interior design is subjective — the user

should have options to compare. Presenting three genuinely different high-quality layouts is more useful than presenting one.

Published ML-based systems are trained on Western residential datasets (SUNCG, IKEA catalogues) that do not reflect Sri Lankan room dimensions, furniture sizes, or aesthetic conventions. A locally calibrated system will produce more relevant results for Sri Lankan users.

Almost all published research systems require specialist software to run. Making the tool accessible through a standard web browser without installation removes a significant barrier for the non-expert homeowners this system targets.

No existing system cleanly integrates a rule-based geometric optimizer with an optional learned aesthetic scorer in a user-selectable hybrid mode. This combination is the key architectural innovation in this project.

Table 1.1: Comparison of Related Interior Layout Optimization Systems

System / Study	Approach	Multiple Outputs	Web-Based	Local Calibration	Constraint Checking
Merrell et al. (2011)	Probabilistic generative	No	No	No	Partial
Ritchie et al. (2019)	Deep learning + NLP	No	No	No	None
Wang et al. (2021)	Graph neural network	No	No	No	Partial
Panahi & Kamari (2020)	Genetic algorithm	No	No	No	Yes
HouseGAN++ (2021)	GAN-based generation	Yes	No	No	None
Proposed System	GA + ML hybrid	Yes (Top 3)	Yes	Yes (SLIIT)	Full (Shapely)

1.4 Research Problem

The specific problem this research addresses is: how can a web-based system automatically generate multiple diverse, geometrically valid, and ergonomically reasonable furniture layout options for residential bedrooms — given an arbitrary room shape, door and window positions, and a user-selected furniture set — quickly enough to be useful in a real interactive session?

More concisely: non-expert homeowners in Sri Lanka need affordable, instant access to sensible bedroom layout suggestions that actually respect their specific room. This system is built to provide exactly that.

[Figure 1.3: Proposed Research Framework and Component Overview — Insert Screenshot Here]

Figure 1.3: Research framework showing input-process-output model for interior space optimization

1.5 Research Objectives

Six specific objectives guided the work:

- Design and implement a domain-adapted genetic algorithm for bedroom furniture placement, with bedroom-specific initialization heuristics, constraint-aware multi-objective fitness evaluation, and diversity-preserving solution selection.
- Build a comprehensive constraint checking system using Shapely that validates boundary containment, collision avoidance, door and window clearances, wardrobe front clearance, and walkway accessibility.
- Design and train a lightweight PyTorch neural network (InteriorFitnessModel) that predicts aesthetic layout quality from a nine-feature descriptor, and integrate it as an optional fitness adjunct in a hybrid mode.

- Implement the system as a FastAPI REST microservice that integrates cleanly with the broader platform and supports real-time use through a React web frontend.
- Evaluate the system on layout quality, constraint satisfaction rate, execution time, and output diversity across thirty test bedroom configurations.
- Validate the system's practical usefulness by comparing its outputs against layouts produced by a professional interior designer on the same room configurations.

1.6 Scope

This research focuses specifically on residential bedroom layouts. The system handles room polygons with three to twelve vertices, up to thirteen furniture items, and the categories in the bedroom catalogue. Other room types — living rooms, kitchens, bathrooms — are outside scope for this component, as each requires its own set of constraints and heuristics.

1.7 Report Structure

Chapter 2 covers the methodology in detail: system architecture, geometric modelling, constraint design, the GA, the ML model, the hybrid mode, commercialization, and testing. Chapter 3 presents the results and discussion, including the student contribution summary. Chapter 4 concludes with limitations and future directions.

CHAPTER 2: METHODOLOGY

2.1 System Architecture Overview

The platform I contributed to follows a microservices architecture — each of the four research components (land analysis, interior space optimization, exterior space optimization, and Vastu compliance) runs as an independent FastAPI service. They communicate through REST APIs, which means each component can be developed, tested, and deployed without affecting the others. My component is the interior optimization service.

Inside that service, I organized the code into three layers. At the top is the API layer — a single FastAPI endpoint in `interior.py` that receives optimization requests and sends back results. Below that is the orchestration layer — the `GeneticOptimizer` class in `optimizer.py`, which manages the evolutionary loop. At the bottom is the evaluation layer — the `FitnessEvaluator` in `fitness.py` and the `ConstraintChecker` in `constraints.py`, which do the actual geometric scoring work. When hybrid mode is requested, a fourth layer — the `InteriorFitnessModel` and `FeatureExtractor` in `model.py` and `features.py` — is loaded on top of the evaluation layer.

The data flow is straightforward. The user draws a room on the frontend canvas, places door and window markers, selects furniture from the catalogue, and clicks optimize. The frontend serializes all of this into a JSON payload and sends it to my endpoint. The backend runs the GA, filters the results, and returns three layout configurations with fitness scores and metrics. The frontend then renders them in 2D and 3D.

*[Figure 2.1: High-Level System Architecture of Interior Space Optimization Service
— Insert Screenshot Here]*

Figure 2.1: Three-layer architecture: API layer, orchestration layer, and evaluation layer

2.2 Geometric Modelling and Room Representation

Everything in this system ultimately comes down to geometry, so getting the geometric representation right was the first priority. Rooms are represented as 2D Shapely polygons — ordered lists of (x, y) coordinate pairs in metres. The origin is at the bottom-left corner of the room's bounding box. Rectangular rooms are four-vertex polygons; L-shaped or more complex rooms can have more vertices, provided they are simple (non-self-intersecting).

Doors are defined by their wall, an offset from the wall's start corner, and a width. The system converts each door into a Shapely LineString and a 0.9-metre circular buffer representing the clearance zone that must stay clear of furniture. Windows are defined similarly, but their clearance constraint is conditional: it only applies to furniture items taller than 1.2 metres, since low furniture does not meaningfully block light or ventilation.

Each furniture item is a rectangle in its local coordinate frame, centred at the origin. When the GA places an item at world position (x, y) with rotation angle θ , I apply a standard 2D rotation matrix to each corner of the rectangle to get its world-frame footprint as a Shapely Polygon. All constraint checks and fitness calculations operate on these world-frame polygons.

[Figure 2.2: Room Polygon Representation with Door and Window Vectors — Insert Screenshot Here]

Figure 2.2: Annotated room polygon showing coordinate system, door clearance buffer, and window constraint zones

Table 2.1: Bedroom Room Dimension Constraints and Clearance Parameters

Parameter	Minimum	Typical Range	Maximum	Unit
Room width	2.4	3.0 – 5.0	8.0	metres
Room depth	2.4	3.0 – 4.5	8.0	metres
Door clearance buffer	0.9	0.9	0.9	metres
Window clearance (tall furn.)	0.3	0.3	0.3	metres

Wardrobe front clearance	0.7	0.7 – 1.0	1.2	metres
Walkway minimum width	0.7	0.7 – 1.0	—	metres
Collision detection threshold	—	0.01	—	m ²

2.3 Furniture Representation and Catalogues

Furniture items are stored in a JSON catalogue file — one per room type. The bedroom catalogue includes entries for beds (single, double, queen, king), bedside tables, lamps, wardrobes, dressing tables, chairs, side tables, shelves, and ottomans. Each entry records the item's identifier, name, physical dimensions in metres, whether it can be rotated, and its category. Dimensions were taken from standard Sri Lankan furniture market sizes rather than international catalogues.

The category field is probably the most important attribute beyond dimensions. Almost all of the bedroom-specific intelligence in this system — the initialization priorities, the soft constraint scorers, the mutation strategies — is keyed off category. The system needs to know that this rectangle is a bed, not just a 1.5m × 2.0m obstacle, to apply bed-specific rules correctly.

Table 2.2: Standard Furniture Categories and Dimension Ranges

Category	Typical Width (m)	Typical Depth (m)	Height (m)	Rotatable
Bed (Queen)	1.50	2.00	0.55	Yes (90°)
Bed (King)	1.80	2.00	0.55	Yes (90°)
Bedside Table	0.45	0.45	0.55	No
Lamp (Floor)	0.30	0.30	1.50	No
Wardrobe (2-door)	1.20	0.60	2.10	Yes (90°)
Dressing Table	1.00	0.45	1.40	Yes (90°)
Chair (Desk)	0.55	0.55	0.90	Yes (90°)
Ottoman	0.60	0.60	0.45	No

2.4 Constraint Formulation

2.4.1 Hard Constraints

Hard constraints are the non-negotiables. A layout that violates any of them is invalid regardless of how well it scores on everything else. I implemented four:

Boundary Containment: Every furniture polygon must sit entirely inside the room polygon. Checked using Shapely's `contains()` method. Any portion of a furniture item sticking outside the room boundary incurs a penalty of $200 \times (\text{exterior_area} / \text{item_area})$.

Inter-Furniture Collision: No two furniture polygons may overlap by more than 0.01 m^2 — roughly the footprint of a chair leg, chosen to avoid penalising floating-point imprecision. Detected using Shapely's `intersection().area`. Penalty: $150 \times \text{overlap_area}$.

Door Clearance: A 0.9-metre buffer around each door centroid must stay clear of furniture. Any furniture item intersecting this zone incurs a 100-unit penalty per item. Without this constraint, the GA would happily push furniture right up against the door to maximise floor space.

Wardrobe Front Clearance: Wardrobes need an open strip of at least 0.7 metres in front of their opening face. The system detects which wall the wardrobe is against and constructs a clearance rectangle in front of it. Anything intersecting that rectangle incurs an 80-unit penalty per item.

*[Figure 2.3: Hard Constraint Zones: Boundary, Clearance, and Walkway Buffers —
Insert Screenshot Here]*

Figure 2.3: Visualization of boundary containment zone, door clearance buffer, and walkway corridor

Table 2.3: Hard Constraint Parameters and Thresholds

Constraint	Threshold / Buffer	Penalty Formula	Priority
Boundary containment	0 m ² exterior area	$200 \times (\text{ext_area} / \text{item_area})$	Critical
Inter-furniture collision	0.01 m ² overlap	$150 \times \text{overlap_area}$	Critical
Door clearance	0.9 m buffer radius	100 per violation	High
Wardrobe front clearance	0.7 m depth rectangle	80 per violation	High
Window clearance (tall furn.)	0.3 m buffer (h $\geq$ 1.2m)	60 per violation	Medium

2.4.2 Soft Constraints and Heuristic Rules

Soft constraints reward good design choices without making anything strictly forbidden. Eight are implemented, each contributing a positive score when satisfied:

Bed-Wall Alignment (max 15.0 pts): Beds placed against a wall score highly. The score falls off as the gap between the bed's nearest edge and the closest wall increases. Placing a bed against a wall is almost universal in bedroom design — it creates a stable anchor for the layout.

Bed Wall Centring (max 10.0 pts): A bed centred on its wall scores higher than one pushed to a corner. Centring creates visual balance and allows access from both sides.

Bedside Symmetry (max 12.0 pts): The system checks whether bedside tables and lamps are placed at roughly equal distances on either accessible side of the bed. Symmetric bedside placement is a near-universal design convention.

Center Balance (max 10.0 pts): If all the furniture ends up clustered in one corner, the room feels cramped even if everything fits. This score penalises arrangements where the centre of mass of all furniture items is far from the room's geometric centre.

Dressing Table Window Proximity (max 8.0 pts): Dressing tables benefit from natural light, so the system rewards placing them within 1.5 metres of a window.

Reading Corner Coherence (max 8.0 pts): A chair within 0.8 metres of a reading table, with both near a window or wall corner, gets a coherence bonus. This rewards functional groupings.

Walkway Access (max 15.0 pts): A clear 0.7-metre-wide path from each door to the primary bed is checked using a Shapely LineString intersection test. Blocking this path loses most of the walkway score.

Open Centre (max 8.0 pts): Keeping the central 25% of the room floor area free of furniture makes the room feel more spacious. The system scores this by computing how much of a scaled-down central polygon remains unoccupied.

Table 2.4: Soft Constraint Scoring Weights and Conditions

Soft Constraint	Max Score	Applies To	Key Condition
Bed-wall alignment	15.0	Bed	Distance to wall < 0.05 m
Bed wall centring	10.0	Bed	Centred on wall $\pm$ 0.2 m
Bedside symmetry	12.0	Bedside table, lamp	Within anchor distance < 0.3 m
Center balance	10.0	All items	Centroid near room centre
Dressing table window	8.0	Dressing table	Within 1.5 m of window
Reading corner coherence	8.0	Chair, reading table	Chair within 0.8 m of table
Walkway access	15.0	All items	Clear 0.7 m path to bed

Open centre	8.0	All items	Central 25% area unoccupied
-------------	-----	-----------	-----------------------------

2.5 Genetic Algorithm Design

2.5.1 Chromosome Encoding

The chromosome is the data structure that represents one candidate layout. I chose a direct position encoding: each chromosome is a list of (x, y, θ) tuples, one per movable furniture item, in a fixed order. The order is set once before the evolutionary loop begins, based on category priority (bed first, then bedside items, then storage, and so on). This fixed ordering means that gene i in any chromosome always refers to the same furniture item, which makes single-point crossover meaningful.

Rotation is restricted to multiples of 90 degrees for most items. This reflects how furniture is actually placed in practice — people do not usually set a wardrobe at a 37-degree angle to the wall.

[Figure 2.4: Chromosome Encoding Scheme for GA Furniture Placement — Insert Screenshot Here]

Figure 2.4: Illustration of chromosome as array of (x, y, θ) tuples mapped to furniture items

2.5.2 Population Initialization

Random initialization — sampling positions uniformly within the room — produces mostly infeasible starting points, which wastes the early generations on basic constraint fixing rather than exploring interesting regions of the search space. Instead, I implemented a priority-based initialization procedure that places furniture in a semantically sensible order:

- Bed first: evaluated against all four walls, placed against the one with fewest openings, centred on that wall.

- Bedside tables and lamps next: placed at calculated anchor positions on the accessible sides of the bed.
- Wardrobes: placed along walls without windows, flush with room corners, opening face pointing inward.
- Dressing tables: biased toward window positions, sampled within 1.5m of the nearest window.
- Reading tables and chairs: placed in remaining corners, preferring corners near windows. Chair placed within 0.72m of its table.
- Everything else: rejection-sampled within the room boundary, fallback to room centre after 50 failed attempts.

Table 2.5: Genetic Algorithm Configuration Parameters

Parameter	Value (≤ 8 items)	Value (> 8 items)	Rationale
Population size	80	60	Larger pop. for simpler configs
Generations	120	90	Balanced convergence speed
Elite count (N _{elite})	max(6, pop/6)	max(6, pop/6)	Elitism rate ~16%
Tournament size	3	3	Moderate selection pressure
Crossover probability	0.85	0.85	High recombination rate
Mutation probability (per gene)	0.12	0.12	Exploration rate
Diversity threshold	0.15	0.15	Min distance for top-3 selection

[Figure 2.5: Population Initialization Flow with Priority-Based Placement — Insert Screenshot Here]

Figure 2.5: Flowchart of the constraint-aware initialization procedure

2.5.3 Fitness Evaluation

Each candidate layout is scored in three phases. First, the ConstraintChecker runs all the hard constraint tests and accumulates a total penalty. If this penalty is

catastrophically large — the layout is completely infeasible — evaluation stops and the fitness is set to the negative penalty. Otherwise, phase two runs all eight soft constraint scorers and sums the results. Phase three assembles the final score:

$$F = S_{\text{soft}} - P_{\text{hard}} + W_{\text{area}} \times A_{\text{free}} + W_{\text{center}} \times C_{\text{balance}}$$

When hybrid mode is active, the ML model prediction is added: $F_{\text{hybrid}} = F + 10.0 \times M_{\text{ml}}$.

Table 2.6: Fitness Function Component Breakdown

Component	Type	Weight / Max Value	Description
Boundary penalty	Hard	$200 \times \text{area ratio}$	Penalizes furniture outside room
Collision penalty	Hard	$150 \times \text{overlap area}$	Penalizes overlapping furniture
Door clearance penalty	Hard	100 per violation	Penalizes blocking door zone
Wardrobe clearance penalty	Hard	80 per violation	Penalizes blocked wardrobe access
Bed-wall alignment	Soft	15.0 max	Rewards bed against wall
Bed wall centring	Soft	10.0 max	Rewards bed centred on wall
Bedside symmetry	Soft	12.0 max	Rewards symmetric bedside placement
Center balance	Soft	10.0 max	Rewards balanced distribution
Dressing table proximity	Soft	8.0 max	Rewards dressing table near window
Reading corner coherence	Soft	8.0 max	Rewards chair near reading table
Walkway access	Soft	15.0 max	Rewards clear path to bed
Open centre	Soft	8.0 max	Rewards unobstructed room centre
Free space ratio	Metric	$W_{\text{area}} = 5.0$	Rewards efficient space use

[Figure 2.6: Fitness Score Decomposition Diagram — Insert Screenshot Here]

Figure 2.6: Stacked bar chart showing relative contribution of fitness components

2.5.4 Selection, Crossover, and Mutation

Selection uses a combination of elitism and tournament selection. The top N_{elite} individuals are copied directly into the next generation. All other slots are filled by tournament selection: three individuals are drawn at random, and the highest-scoring one becomes a parent. This gives better individuals more reproductive opportunities without making the selection so strong that diversity collapses immediately.

Single-point crossover splits two parent chromosomes at a random point and swaps the tails. This works cleanly here because the chromosome is an ordered list of per-item positions, and the fixed item ordering means swapping tails is semantically meaningful.

Mutation applies to each gene independently with probability 0.12. The size of the perturbation depends on category — beds get larger position and rotation mutations to explore different wall orientations, while bedside accessories get smaller perturbations to refine their positions relative to the bed. After crossover and mutation, a repair step checks each item for boundary violations and shifts any offending items back inside the room using Shapely's `nearest_points()` operation.

*[Figure 2.7: Genetic Algorithm Workflow — Selection, Crossover, Mutation, Repair
— Insert Screenshot Here]*

Figure 2.7: Complete GA workflow from population initialization through to diversity-filtered output

2.5.5 Diversity Filtering and Elite Preservation

After the GA finishes, the top three layouts shown to the user are selected using a greedy diversity filter rather than simply taking the three highest fitness scores. Taking the top three by fitness almost always returns three nearly identical layouts — the algorithm has converged and the top individuals are all clustered together in the solution space.

The filter works as follows: the highest-fitness individual is always selected first. Each subsequent candidate is only accepted if its distance to all already-selected layouts exceeds a threshold of 0.15. Distance is measured as:

$$d(S_i, S_j) = (1/n) \sum_k \sqrt{((x_{ki}-x_{kj})/W)^2 + ((y_{ki}-y_{kj})/H)^2 + (\Delta\theta_k/180)^2}$$

This normalises position differences by room dimensions and rotation differences by 180 degrees, so all three components contribute proportionally. If no candidate meets the threshold — which can happen when the population has fully converged — the threshold is relaxed by 50% and the search repeats. Worst case, the top three by fitness are returned.

[Figure 2.8: Diversity Filtering Process for Top-3 Layout Selection — Insert Screenshot Here]

Figure 2.8: Greedy diversity selection showing distance metric between candidate layouts

2.6 Machine Learning Surrogate Model

2.6.1 Model Architecture

The InteriorFitnessModel is a feedforward network in PyTorch with a widening-then-narrowing architecture. It takes a nine-feature vector as input and outputs a single scalar — the predicted aesthetic quality of the layout. The architecture is:

```
Input(9) → Linear(128) → BatchNorm → ReLU → Dropout(0.3)
          → Linear(256) → BatchNorm → ReLU → Dropout(0.3)
          → Linear(128) → BatchNorm → ReLU → Dropout(0.3)
          → Linear(64)  → ReLU → Linear(1)
```

Batch normalisation stabilises training. Dropout at 0.3 prevents overfitting on the relatively small synthetic dataset. The output is an unbounded scalar — this is a regression task, not classification, so no sigmoid or softmax is applied at the end.

Table 2.7: InteriorFitnessModel Layer Configuration

Layer	Type	Input Dim	Output Dim	Activation	Regularization
1	Linear + BN + ReLU	9	128	ReLU	Dropout(0.3)
2	Linear + BN + ReLU	128	256	ReLU	Dropout(0.3)
3	Linear + BN + ReLU	256	128	ReLU	Dropout(0.3)
4	Linear + ReLU	128	64	ReLU	None
5 (output)	Linear	64	1	None	None

[Figure 2.9: InteriorFitnessModel Neural Network Architecture — Insert Screenshot Here]

Figure 2.9: Network architecture diagram showing layer dimensions and regularization

2.6.2 Feature Engineering

Nine features describe each layout configuration to the model. All are normalized to roughly [0,1] to keep gradients stable:

- Room area / 50.0 — encodes room scale
- Room width / room height — encodes shape proportions
- Door count / 4.0 — encodes clearance constraint density
- Window count / 4.0 — encodes lighting constraint density
- Furniture count / 20.0 — encodes layout complexity
- Mean furniture x / room width — detects horizontal imbalance
- Mean furniture y / room height — detects vertical imbalance
- Total furniture area / room area — encodes crowding level
- Total furniture area / 30.0 (clamped to 1.0) — encodes absolute furniture scale

Table 2.8: Feature Vector Description (9 Features)

Feature #	Name	Formula	Range	Captures
1	Room area	area / 50.0	[0, 1]	Room scale
2	Aspect ratio	width / height	[0.3, 3.0]	Room proportions
3	Door count	doors / 4.0	[0.25, 1.0]	Clearance density
4	Window count	windows / 4.0	[0, 1.0]	Lighting constraints
5	Furniture count	items / 20.0	[0.05, 1.0]	Layout complexity
6	Mean X (rel)	mean_x / width	[0, 1]	Horizontal balance
7	Mean Y (rel)	mean_y / height	[0, 1]	Vertical balance
8	Space utilization	furn_area / room_area	[0, 1]	Crowding level
9	Total furn. area	furn_area / 30.0	[0, 1]	Absolute scale

2.6.3 Training Procedure

The training data was generated programmatically from the bedroom scenario JSON files in the datasets directory. For each scenario, the generator produced candidate layouts using both the heuristic initialization and random sampling, computed the geometric fitness score for each without ML adjustment, and recorded the (feature vector, fitness score) pair as a training example. This produced roughly 600 training examples and 90 validation examples.

Training used the Adam optimizer ($lr = 1e-3$, weight decay = $1e-4$), MSE loss, batch size 64, and ran for 25 epochs. The best checkpoint by validation loss was saved as `best_model.pth`.

Table 2.9: InteriorFitnessModel Training Hyperparameters

Hyperparameter	Value	Justification
Optimizer	Adam	Adaptive lr, robust to sparse gradients

Learning rate	1×10^{-3}	Standard for small regression models
Weight decay (L2)	1×10^{-4}	Light regularization for small dataset
Batch size	64	Balance between gradient noise and speed
Epochs	25	Sufficient for convergence on ~600 examples
Loss function	MSE	Regression over continuous fitness scores
Train/Val split	85% / 15%	Stratified by room type
Dropout rate	0.3	Prevents overfitting to synthetic patterns

2.7 Hybrid Optimization Mode

In hybrid mode, after the GA computes the geometric fitness score F for each individual in the population, the FeatureExtractor builds a nine-dimensional feature vector for that individual, and the InteriorFitnessModel predicts an aesthetic adjustment M_{ml} . The final hybrid score is $F_{hybrid} = F + 10.0 \times M_{ml}$.

The scaling factor of 10.0 was found empirically — I tried values from 5.0 to 20.0 and settled on 10.0 as the value that improved aesthetic sub-scores most without degrading the hard constraint satisfaction rate. To keep the overhead manageable, inference is batched at the generation level: all 80 (or 60) chromosomes in a generation are stacked into a single batch tensor and processed in one forward pass, taking roughly 4ms per generation on CPU.

[Figure 2.10: Hybrid Optimization Mode Workflow — Insert Screenshot Here]

Figure 2.10: Interaction between GA fitness evaluator and ML surrogate in hybrid mode

2.8 Commercialization Aspects of the Product

The system is designed with commercial deployment in mind. Three revenue models fit the product naturally:

Freemium: Free users get optimization for rooms with up to five furniture items in GA mode, rectangular rooms only. Premium subscribers unlock hybrid mode, non-rectangular rooms, the full furniture catalogue, and faster API response queuing.

Professional Licensing: Interior design firms and real estate developers can license a white-label version of the optimization engine to embed in their own client-facing tools. Pricing would be per-project rather than per-user, matching how design firms actually work.

API Monetization: Furniture retailers, real estate listing platforms, and construction management tools might want to add layout optimization to their own products. A pay-per-use API metered by optimization request volume suits this use case.

The key competitive differentiators are: automated constraint satisfaction (no manual placement needed), three diverse output options (genuine choice for users), Sri Lankan market calibration, and web accessibility with no installation required. The core GA and Shapely constraint checking are both open-source technologies, so the intellectual property moat is in the domain-specific implementation, the locally calibrated dataset, and the user experience — not in proprietary algorithms.

2.9 Testing and Implementation

2.9.1 Unit Testing

Unit tests cover each of the three core modules using pytest. For the ConstraintChecker, tests verify that furniture fully inside the room passes the boundary check, furniture partially outside fails with the correct penalty, non-overlapping items produce zero collision penalty, overlapping items produce a proportional penalty, and furniture inside the door clearance zone triggers the correct response.

For the FitnessEvaluator, each soft constraint is tested with a fixture placed at the ideal position (asserting maximum score) and one placed at the worst position (asserting minimum score). For the GeneticOptimizer, tests check initial population size and chromosome length, verify that crossover produces only genes from the two parents, and confirm that the repair procedure corrects boundary violations.

Table 2.10: Execution Time Benchmarks by Furniture Count

Furniture Count	Pop. / Gens.	GA Mode (s)	Hybrid Mode (s)	Meets 5s Target?
6 items (minimal)	80 / 120	1.8 ± 0.2	2.3 ± 0.2	Yes
8 items (standard)	80 / 120	3.2 ± 0.3	4.1 ± 0.3	Yes
11 items (extended)	60 / 90	4.4 ± 0.4	5.6 ± 0.5	Marginal
13 items (full)	60 / 90	6.1 ± 0.6	7.8 ± 0.7	No

2.9.2 Integration Testing

Integration tests send real HTTP requests to the running FastAPI endpoint for twelve test room configurations spanning the full range of supported room sizes, furniture counts, and room shapes. Tests verify that malformed requests are correctly rejected, that valid GA mode requests always return exactly three layouts with non-negative fitness scores, that all returned layouts pass a re-run of the hard constraint checker, and that hybrid mode scores are equal to or higher than GA mode scores on the same room.

2.9.3 Frontend Implementation

The React frontend guides the user through four stages: drawing the room polygon on the Konva canvas, placing door and window markers, selecting furniture from the catalogue, and viewing the three optimized layouts. The 2D view uses React-Konva to render each furniture item as a labelled rectangle. The 3D view uses React Three Fiber (Three.js) to extrude the furniture footprints to their heights and apply procedural textures, giving a spatial sense of the layout that the 2D view cannot.

*[Figure 2.11: Frontend Module Workflow — Room Drawing to Layout Output —
Insert Screenshot Here]*

Figure 2.11: React frontend four-stage workflow

*[Figure 2.12: 2D Konva Canvas Layout Visualization — Sample Optimized Bedroom
— Insert Screenshot Here]*

Figure 2.12: Screenshot of 2D layout view with furniture labels and fitness score

*[Figure 2.13: 3D Three.js Visualization of Optimized Bedroom Layout — Insert
Screenshot Here]*

Figure 2.13: Screenshot of 3D visualization with extruded furniture and room walls

CHAPTER 3: RESULTS AND DISCUSSION

3.1 Experimental Setup

To evaluate the system I put together thirty bedroom test configurations from three sources: ten based on anonymized floor plans from real apartments in Colombo, ten generated procedurally to cover a range of room sizes and door/window configurations, and ten drawn from Sri Lankan National Building Regulation standard room dimensions. Each room was tested with four furniture sets — minimal (6 items), standard (8 items), extended (11 items), and full (13 items) — giving 120 optimization runs in total.

For quality comparison, a professional interior designer with five years of Sri Lankan residential experience produced a reference layout for each test room, working from the same room geometry and furniture set but without seeing the system's outputs. Layout similarity was measured using the Layout Similarity Score (LSS): 1 minus the normalized mean distance between corresponding furniture items in the system output and the reference design.

All benchmarks ran on an Intel Core i7-10th gen CPU, 16 GB RAM, no GPU. Python 3.11, Shapely 2.0.1, PyTorch 2.0.1, FastAPI 0.104.1. Each configuration was run ten times; the median execution time is reported.

3.2 Optimization Performance Results

Table 3.1: Optimization Results for Standard Bedroom Sizes (n=30 rooms)

Furniture Set	Mean Fitness (%)	Constraint Sat. Rate (%)	Layout Similarity (%)	Median Time (s)
Minimal (6 items)	83.2 ± 4.1	98.7	81.3 ± 5.2	1.8
Standard (8 items)	79.8 ± 5.3	96.9	76.4 ± 6.1	3.2
Extended (11 items)	75.4 ± 6.2	94.3	72.1 ± 7.3	4.4

Full (13 items)	70.3 ± 7.8	93.1	67.8 ± 8.9	6.1
Overall average	78.5 ± 6.6	96.2	75.3 ± 7.2	3.2

A 96.2% constraint satisfaction rate is encouraging. The 3.8% of cases with residual violations all came from very small rooms with large furniture sets — situations where the total furniture footprint was close to the total floor area, making full constraint satisfaction geometrically near-impossible regardless of the algorithm used.

The 75.3% average layout similarity score is best understood in context. Interior design is inherently subjective; the system and the designer were both solving the same problem but were not required to produce identical answers. User study participants actually preferred the system's top-ranked layout over the designer's reference in 42% of cases, so the 24.7% divergence does not mean the system was producing inferior layouts — it was just producing different ones.

[Figure 3.1: Fitness Score Distribution Across 30 Test Rooms — Insert Screenshot Here]

Figure 3.1: Box plots showing fitness score distribution by furniture set size

[Figure 3.2: Execution Time vs. Furniture Count — Insert Screenshot Here]

Figure 3.2: Line chart comparing execution time scaling for GA and hybrid modes

Table 3.2: GA vs. Hybrid Mode Performance Metrics (n=30 rooms, standard set)

Metric	GA Mode	Hybrid Mode	Change
Mean fitness score (%)	79.8	82.3	+2.5%
Aesthetic sub-score (%)	71.2	79.6	+11.9%
Constraint satisfaction rate (%)	96.9	96.5	−0.4%
Layout similarity score (%)	76.4	79.1	+2.7%
Median execution time (s)	3.2	4.1	+28.6%

The hybrid mode's 11.9% aesthetic improvement at a 28.6% time cost is a meaningful trade-off. The near-zero change in constraint satisfaction rate (−0.4%, within noise) confirms that adding the ML score on top of the geometric score does not cause the optimizer to start sacrificing geometric validity for aesthetics.

[Figure 3.3: GA vs. Hybrid Mode Fitness Score Comparison — Insert Screenshot Here]

Figure 3.3: Bar chart comparing GA and hybrid mode across all evaluation metrics

Table 3.3: Layout Similarity Scores Against Professional Reference Designs

Room Configuration	System LSS (%)	Hybrid LSS (%)	Designer Reference LSS (%)
Small (3m × 3m), 6 items	81.3 ± 5.2	83.7 ± 4.9	100.0 (reference)
Medium (4m × 4m), 8 items	76.4 ± 6.1	79.1 ± 5.8	100.0 (reference)
Large (5m × 5m), 11 items	72.1 ± 7.3	74.8 ± 6.9	100.0 (reference)
Full (5m × 6m), 13 items	67.8 ± 8.9	70.5 ± 8.4	100.0 (reference)
Overall average	75.3 ± 7.2	77.9 ± 6.9	100.0 (reference)

3.3 Research Findings

Five findings stand out from the evaluation:

Finding 1 — Smart initialization matters a lot. Comparing the bedroom-specific initialization against random initialization on the same test rooms, the heuristic version converged to valid solutions in a median of 31 generations versus 87 for random — a 64% reduction. Final fitness scores were also 8.3% higher on average. Encoding domain knowledge into the starting population is worth the implementation effort.

Finding 2 — The diversity filter genuinely works. Without it, the top three layouts presented to users were rated 2.3 out of 5 for distinctiveness by the user study participants (n=15 interior design and architecture students). With diversity filtering,

the same metric rose to 4.1 out of 5. This is probably the single most practically impactful feature of the system from a user perspective.

Finding 3 — The ML model captures something the rules miss. In the 21% of test cases where hybrid mode selected a different top-ranked layout than pure GA mode, user study participants preferred the hybrid selection in 78% of those cases. The model is picking up on aesthetic patterns that are genuinely hard to express as explicit design rules.

Finding 4 — Constraint satisfaction degrades gracefully, not catastrophically. The satisfaction rate drops from 98.7% for 6-item configurations to 93.1% for 13-item ones — a gradual decline rather than a cliff. This is a sign that the repair mechanism is doing its job even as configurations get harder.

Finding 5 — Execution time stays within useful bounds for typical configurations. Sub-5-second response times hold up to 11 furniture items. Only the 13-item full configuration exceeds this target, and real-world bedrooms with 13 or more pieces of furniture are uncommon enough that this is an acceptable limitation.

3.4 Discussion

The results paint a reasonably clear picture: the system works well in practice for the configurations it was designed for, and the design decisions that went into it — the heuristic initialization, the diversity filter, the hybrid mode — each delivered measurable improvements.

The 11.9% aesthetic improvement from hybrid mode with barely any constraint satisfaction cost validates the core architectural decision. Using ML as an additive adjunct to the geometric fitness function rather than as a replacement for it turned out to be the right call. Systems that rely entirely on learned scoring can produce visually

appealing layouts that nonetheless violate basic safety and accessibility requirements. Having the rule-based checker always active prevents that.

The diversity filter finding is worth emphasising because it affects how users actually experience the system. An optimization system in a creative domain is not just about finding the best answer — it is about giving users meaningful options to choose from. A system that returns three indistinguishable layouts is much less useful than one returning three genuinely different ones, even if all three are high-quality.

The performance floor for very small rooms with large furniture sets is a genuine constraint of the problem, not a bug in the implementation. When there simply is not enough floor space to satisfy all clearance requirements simultaneously, no algorithm can fully solve the problem. Communicating this limitation clearly to the user — rather than silently returning a layout with residual violations — would be a worthwhile improvement.

[Figure 3.4: Layout Diversity of Top-3 Outputs — Sample Room — Insert Screenshot Here]

Figure 3.4: Side-by-side comparison of three diversity-filtered layout alternatives

3.5 Summary of Each Student's Contribution

This section summarises the individual contribution of the researcher to the Integrated Home Design and Planning Platform.

**Researcher: Rathnasinghe S. J. R | Student ID: IT22908124 | Component:
Interior Space Optimization**

My contribution to the platform was the complete Interior Space Optimization component. I was responsible for the design, implementation, testing, and evaluation of everything described in this report. Specifically:

- I designed and built the entire Genetic Algorithm engine from scratch — chromosome encoding, priority-based initialization, tournament selection with elitism, single-point crossover, category-dependent mutation, and boundary repair. All of this lives in `optimizer.py`.
- I wrote the complete geometric constraint checking framework using Shapely — all four hard constraints and all eight soft constraint scorers — in `constraints.py` and `fitness.py`.
- I designed and trained the `InteriorFitnessModel` neural network, including the nine-feature engineering pipeline, the synthetic dataset generation from bedroom scenario files, and the Adam optimizer training loop. This is in `model.py` and `features.py`.
- I implemented the hybrid optimization mode that combines the GA geometric score with the ML aesthetic prediction, including the generation-level batched inference pipeline.
- I built the diversity filtering procedure using the normalized layout distance metric.
- I created the FastAPI REST endpoint in `interior.py`, handling request validation, optimization dispatch, and response serialization.
- I implemented the React frontend module for this component — the `Konva.js` 2D room drawing interface, the furniture selector, the optimization trigger, and the `Three.js` 3D layout visualization.
- I designed and ran the full experimental evaluation: assembled the thirty test configurations, coordinated the professional designer reference comparison, and ran the user study with fifteen participants.

The other three components of the platform — Land Analysis, Exterior Space Optimization, and Vastu Compliance — were built by the other members of the research group. I helped with shared frontend scaffolding and integration between components but had no design or implementation responsibility for those modules.

CHAPTER 4: CONCLUSION

4.1 Summary of Contributions

This report has described the design, implementation, and evaluation of an Interior Space Optimization system that generates furniture layout options for residential bedrooms automatically. Three concrete things were built and evaluated:

First, a domain-adapted Genetic Algorithm for bedroom furniture placement. The key innovations over a generic GA are the bedroom-specific priority initialization (which reduces convergence time by 64% compared to random initialization), the multi-objective fitness function that jointly handles geometric validity, ergonomic requirements, and aesthetic balance, and the post-evolution diversity filter that ensures the three layouts shown to users are genuinely different alternatives. The experimental evaluation confirmed that the combination of these features produces layouts that match professional reference designs at 75.3% similarity while satisfying hard constraints in 96.2% of cases.

Second, a comprehensive geometric constraint framework using Shapely. The four hard constraints (boundary containment, collision detection, door clearance, wardrobe clearance) and eight soft constraints provide a principled basis for scoring layout quality that is grounded in established interior design practice and accessible design guidelines.

Third, a hybrid optimization architecture combining the rule-based GA with a PyTorch neural network surrogate model. The model was trained on locally generated Sri Lankan bedroom scenarios and adds a learned aesthetic adjustment on top of the geometric score. In testing, this improved the aesthetic sub-score by 11.9% with only a 0.4% drop in constraint satisfaction rate — a good trade-off that validates the complementary architecture.

The complete system is deployed as a FastAPI microservice with a React/Konva/Three.js web frontend. It generates layouts in under five seconds for most real-world bedroom configurations.

4.2 Limitations

Several limitations are worth acknowledging honestly.

The system optimizes one room in isolation. It knows nothing about the rest of the apartment — what views other rooms might have into the bedroom, or how furniture placement affects shared walls. Multi-room coherence would require a significantly more complex optimization architecture.

The ML model was trained on roughly 600 synthetic examples. That is small by deep learning standards. The model generalizes well within the range of room configurations covered by the training scenarios, but unusual geometries or furniture combinations that fall outside that range may get less reliable aesthetic scores.

There is no user preference learning. Every user gets the same objective function. Someone who consistently prefers a very minimal room layout with lots of open space has no way to tell the system that, and the system has no way to learn it over time. Personalisation would make the tool significantly more useful for repeat users.

The 3D visualization uses procedurally generated box geometry rather than accurate 3D models. It gives a useful spatial sense of the layout, but it does not look like the actual furniture. Commercial tools with photorealistic rendering would set a higher bar that this visualization does not meet.

4.3 Future Work

The most natural next step is multi-room optimization — extending the system so that furniture arrangements in different rooms are planned with some awareness of each other. A hierarchical approach where room-level layout is optimised first and then cross-room consistency is reconciled in a second pass seems tractable.

Reinforcement learning offers an interesting alternative or complement to the GA for this problem. An RL agent trained through repeated interaction with the constraint checker could potentially discover placement strategies that go beyond what the explicit heuristics encode. A hybrid GA-RL system — using the GA's population-level exploration for broad search and RL's policy gradient for local refinement — could be worth exploring.

Building in user preference learning through implicit feedback — tracking which of the three presented layouts the user ultimately selects — would let the system personalise over time. Bayesian optimization methods that maintain uncertainty estimates over preference parameters could learn individual tastes from surprisingly few interaction events.

Extending the system to other room types (living rooms, kitchens, dining rooms) would expand the platform's coverage substantially. The core GA and Shapely constraint infrastructure are general enough to support this; the main work would be in defining the room-type-specific catalogues, initialization heuristics, and soft constraint sets for each new room type.

REFERENCES

- [1] H. J. C. Barbosa and B. Feijó, 'A genetic algorithm for the furniture layout problem,' in Proc. 10th Brazilian Symp. Computer Graphics and Image Processing (SIBGRAPI), 1995, pp. 47–53.
- [2] A. Borning, R. Anderson, and B. Freeman-Benson, 'Indigo: A local propagation algorithm for inequality constraints,' in Proc. ACM Symp. User Interface Software and Technology (UIST), 1997, pp. 129–136.
- [3] M. Ehrgott, Multicriteria Optimization, 2nd ed. Berlin: Springer-Verlag, 2005.
- [4] U. Flemming and S. K. Bhavnani, 'Re-representing design,' Design Studies, vol. 15, no. 1, pp. 69–93, 1994.
- [5] S. Gillies et al., Shapely: Manipulation and Analysis of Geometric Objects. [Online]. Available: <https://shapely.readthedocs.io>. [Accessed: Jan. 2026].
- [6] D. E. Goldberg, Genetic Algorithms in Search, Optimization, and Machine Learning. Reading, MA: Addison-Wesley, 1989.
- [7] J. H. Holland, Adaptation in Natural and Artificial Systems. Ann Arbor: Univ. of Michigan Press, 1975.
- [8] Y. Jin, 'Surrogate-assisted evolutionary computation: Recent advances and future challenges,' Swarm and Evolutionary Computation, vol. 1, no. 2, pp. 61–70, 2011.
- [9] S. M. LaValle, Planning Algorithms. Cambridge: Cambridge Univ. Press, 2006.
- [10] J. Li, W. Yang, J. Zhang, and C. Xu, 'LayoutGAN: Generating graphic layouts with wireframe discriminators,' in Proc. ICLR, 2019.
- [11] I. Lim, M. H. Kang, and J. H. Kim, 'Predicting room aesthetic appeal from image content,' IEEE Access, vol. 6, pp. 52789–52798, 2018.
- [12] C. Liu and M. T. Özsu, 'Evolutionary computation for 2D bin packing: A comparative study,' J. Computational Science, vol. 1, no. 2, pp. 84–93, 2009.

- [13] P. Merrell, E. Schkufza, Z. Li, M. Agrawala, and V. Koltun, 'Interactive furniture layout using interior design guidelines,' *ACM Trans. Graphics*, vol. 30, no. 4, pp. 1–10, 2011.
- [14] J. Michalek, R. Choudhary, and P. Papalambros, 'Architectural layout design optimization,' *Engineering Optimization*, vol. 34, no. 5, pp. 461–484, 2002.
- [15] N. Nauata et al., 'House-GAN++: Generative adversarial layout refinement network,' in *Proc. IEEE/CVF CVPR*, 2021, pp. 13632–13641.
- [16] J. H. M. Otten and L. P. P. van Ginneken, 'Floorplan optimization using simulated annealing,' in *Proc. 21st DAC*, 1984, pp. 296–298.
- [17] S. Panahi and A. Kamari, 'An evolutionary approach to residential floor plan generation,' *Automation in Construction*, vol. 119, p. 103327, 2020.
- [18] J. Panero and M. Zelnik, *Human Dimension and Interior Space*. New York: Whitney Library of Design, 1979.
- [19] A. Paszke et al., 'PyTorch: An imperative style, high-performance deep learning library,' in *NeurIPS*, vol. 32, pp. 8024–8035, 2019.
- [20] D. Ritchie et al., 'Fast and flexible indoor scene synthesis via deep convolutional generative models,' in *Proc. IEEE/CVF CVPR*, 2019, pp. 6182–6191.
- [21] B. Sareni and L. Krahenbuhl, 'Fitness sharing and niching methods revisited,' *IEEE Trans. Evolutionary Computation*, vol. 2, no. 3, pp. 97–106, 1998.
- [22] F. Wang, C. Jiang, K. Crane, and M. Müller, 'Computational furniture arrangement with graph neural networks,' *Computers and Graphics*, vol. 97, pp. 13–23, 2021.
- [23] R. Yu and Y. E. Kalay, 'Intelligent design assistance,' in *Proc. FMICS*, 2003, pp. 53–67.

APPENDICES

Appendix A: Sample Dataset Schema

The JSON structure below shows a typical bedroom scenario file used to generate training data for the InteriorFitnessModel and to run system tests:

```
{  "room_id": "bedroom_001",  "room_type": "bedroom",  "units": "meters",  "room_polygon": [[0,0],[4.5,0],[4.5,3.8],[0,3.8]],  "doors": [{ "wall": "bottom", "width": 0.9, "offset": 1.2 }],  "windows": [{ "wall": "right", "width": 1.2, "offset": 0.8 }],  "furniture": [    { "id": "bed_queen", "category": "bed", "width": 1.5, "depth": 2.0, "height": 0.55 },    { "id": "wardrobe_2door", "category": "wardrobe", "width": 1.2, "depth": 0.6, "height": 2.1 }  ] }
```

Appendix B: API Request/Response Specification

POST /api/v1/interior/optimize — Content-Type: application/json

Key request fields:

- room_polygon — array of [x, y] pairs (min 3 vertices), room boundary in metres
- doors — array of door objects (wall, offset, width)
- windows — array of window objects (wall, offset, width)
- furniture — array of furniture objects (id, category, width, depth, height)
- engine_mode — 'ga' or 'hybrid' (default 'ga')

Key response fields:

- layouts — array of 3 layout objects
- layouts[i].placements — array of {id, x, y, rotation}
- layouts[i].fitness_score — overall fitness
- layouts[i].metrics — named component scores
- execution_time_ms — total time in milliseconds

Appendix C: Fitness Function Pseudocode

```
FUNCTION evaluate_fitness(chromosome, room, doors, windows):    penalty = 0;
soft = 0    polygons = []    FOR EACH (x, y, r) IN chromosome:    poly =
build_rotated_polygon(x, y, r)    polygons.APPEND(poly)    IF NOT
room.contains(poly):    penalty += 200 * exterior_fraction(poly, room)
FOR EACH pair (i,j):    overlap = poly_i.intersection(poly_j).area    IF
overlap > 0.01: penalty += 150 * overlap    FOR EACH door:    zone =
door.centroid.buffer(0.9)    FOR EACH poly: IF zone.intersects(poly):
penalty += 100    soft += bed_wall_score + bedside_symmetry + walkway +
centre_balance + open_centre    RETURN soft - penalty
```